EXPERIMENT-4:

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Code:

```
# ================================================

# Artificial Neural Network using Backpropagation

# Breast Cancer Wisconsin Dataset

# ================================================


# Import Libraries

import pandas as pd

import numpy as np

from google.colab import files


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report


import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense


# Upload Dataset

uploaded = files.upload()


# Read Dataset

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)


print("First 5 Rows")
```

```
print(df.head())

# Remove unnecessary columns
if 'id' in df.columns:
    df.drop('id', axis=1, inplace=True)

if 'Unnamed: 32' in df.columns:
    df.drop('Unnamed: 32', axis=1, inplace=True)

# Features and Target
X = df.drop('diagnosis', axis=1)
y = df['diagnosis']

# Encode Target
encoder = LabelEncoder()
y = encoder.fit_transform(y)   # M=1, B=0

# Feature Scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Build ANN
```

```
model = Sequential()

model.add(Dense(16, activation='relu', input_shape=(X_train.shape[1],)))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

# Compile
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train
model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=16,
    verbose=1
)

# Predict
y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5).astype(int)

# Accuracy
print("\nAccuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")

# Confusion Matrix
```

```
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report")
print(classification_report(y_test, y_pred))

# Predict New Sample
sample = X_test[0].reshape(1, -1)
prediction = model.predict(sample)

if prediction[0][0] > 0.5:
    print("\nPredicted Diagnosis: Malignant (Cancer)")
else:
    print("\nPredicted Diagnosis: Benign (No Cancer)")
```

Output:

```
Accuracy: 94.74 %

Confusion Matrix
[[67  4]
 [ 2 41]]

Classification Report
              precision    recall  f1-score   support

           0       0.97      0.94      0.96        71
           1       0.91      0.95      0.93        43

    accuracy                           0.95       114
   macro avg       0.94      0.95      0.94       114
weighted avg       0.95      0.95      0.95       114

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step

Predicted Diagnosis: Benign (No Cancer)
```